

Svelte **Review Cards**



Eight concepts an interviewer can probe and a beginner must own.
Read the question, cover the answer, answer aloud, then check the
code and the figure.

19 — Key blocks: rebuilding DOM on demand

KEY BLOCKS DOM COMPONENTS

Q. What does the `{#key}` block do in Svelte?

ANSWER. The `{#key}` block watches an expression. Whenever that expression changes, Svelte destroys everything inside the block and recreates it from scratch. Components inside are freshly instantiated, their state starts over, and transitions play again.

WHY IT WORKS. Svelte uses the key to decide identity. Same key means same content, patched as usual. New key means new content, so the old DOM and component instances are thrown away and rebuilt.

ArticlePage.svelte

```
<script>
  let { article } = $props();
</script>

{#key article.id}
  <ArticleView {article} />
{/key}
```

WHAT `{#KEY}` DOES

key = article.id

WHEN THE VALUE CHANGES



block contents

destroyed and recreated · transitions replay

i Summary. When the key expression changes, Svelte throws away everything inside the block

and rebuilds it. Use `{#key}` when you need a guaranteed fresh start.

19 — Key blocks: a key change never updates, it rebuilds

KEY BLOCKS STATE COMPONENTS

Q. I switched articles and my half-typed comment draft vanished — I only changed the key, so why did everything inside rebuild?

ANSWER. Because `{#key}` never updates anything. When the key expression changes, Svelte destroys everything inside the block — the component instance, its local state, your draft — and builds a brand-new instance from zero. A key change is not a refresh; it is a teardown followed by a fresh start.

WHY IT WORKS. Every normal state change in Svelte patches the DOM in place and preserves component state, so it is natural to expect a key change to behave the same way. But the key exists to declare identity: a new key means “this is a different thing,” and Svelte takes you literally, throwing the old instance away so no stale state leaks into the new one.

ArticlePage.svelte

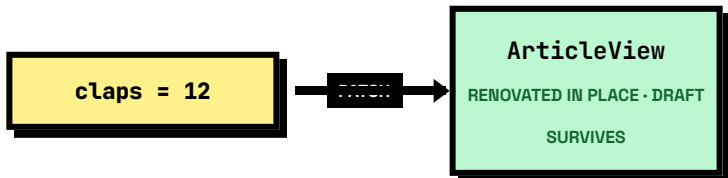
```
<script>
  let { article } = $props();
</script>

{#key article.id}
  <ArticleView {article} />
  <!-- new id → old instance destroyed, fresh state,
transitions replay -->
{/key}
```

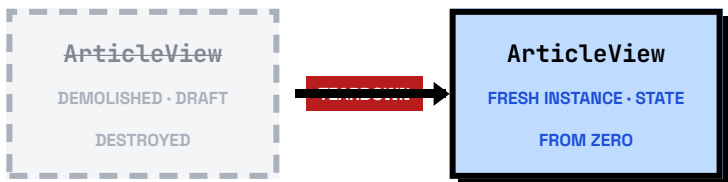
DIAGRAM CONCEPT. The Wrecking Ball: a normal state change is a renovation — the building stays, the furniture stays; a key change is a demolition at the same address — old instance razed, new one erected, nothing inside survives.

THE WRECKING BALL

ID STAYS 7 • NORMAL UPDATE



ID GOES 7 → 8 · KEY FIRES



- ① **Summary.** A key change is a demolition, not a refresh: everything inside `{#key}` is destroyed and rebuilt with fresh state. Reach for it when switching to a new article should feel like opening a clean page — and expect anything unsaved inside to be gone.